

Функциональный стиль в Kotlin

Лекция 2: Коллекции, цепочки вызовов, безопасная обработка, scope-функции и Pair/Triple

Александр Глускер

Курс «Мобильная разработка»

10 февраля 2026 г.

Содержание

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takeIf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple
- 8 Практические примеры

Цели лекции

- Научиться писать в функциональном стиле
- Освоить цепочки вызовов без var и mutable коллекций
- Уметь безопасно обрабатывать ошибки
- Подготовиться к семинару

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

Функциональный стиль — суть

- **Функции как объекты первого класса:** можно передавать, возвращать, хранить
- **Иммутабельность:** данные не изменяются — создаются новые структуры
- **Чистые функции:** одинаковые входы → одинаковые выходы, без побочных эффектов
- **Выражения вместо операторов:** почти всё возвращает значение
- **Декларативность:** описываем *что*, а не *как*

В Kotlin

Поддержка через лямбды, расширения, стандартную библиотеку коллекций, scope-функции.

Почему функциональный стиль?

- Читаемость: цепочки вызовов выражают намерение
- Безопасность: меньше ошибок, особенно с null и состоянием
- Композируемость: легко комбинировать операции
- Тестируемость: чистые функции проще тестировать
- Поддержка многопоточности: иммутабельность → нет гонок

Важно

Не догма — Kotlin позволяет смешивать парадигмы. Используйте функциональный стиль там, где он уместен.

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map**
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

List — упорядоченная коллекция

```
1 val numbers = listOf(1, 2, 3, 4, 5)
2 val empty = emptyList<Int>()
3
4 // Индексация
5 println(numbers[0]) // 1
6
7 // Доступ к первому и последнему /
8 println(numbers.first()) // 1
9 println(numbers.last()) // 5
```

Set — уникальные элементы без порядка

```
1 val unique = setOf(1, 2, 3, 2, 1) // {1, 2, 3}
2 val emptySet = emptySet<Int>()
3
4 // Проверка наличия
5 println(unique.contains(2)) // true
6 println(2 in unique)        // true
7
8 // Операции над множествами
9 val a = setOf(1, 2, 3)
10 val b = setOf(3, 4, 5)
11 println(a intersect b) // {3}
12 println(a union b)     // {1, 2, 3, 4, 5}
```


Map — пары ключ-значение

```
1 val map = mapOf("a" to 1, "b" to 2, "c" to 3)
2 val emptyMap = emptyMap<String, Int>()
3
4 // Получение значения
5 println(map["a"])           // 1 может( быть null)
6 println(map.getValue("a"))  // 1 бросит( исключение, если нет )
7 println(map.getOrElse("d", 0)) // 0
8
9 // Итерация
10 map.forEach { (key, value) ->
11     println("$key: $value")
12 }
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления**
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

Зачем Sequence?

Проблема: при работе с большими коллекциями, каждый вызов `map/filter` создает промежуточный список.

```
1 val result = (1..1000000)
2   .map { it * 2 }      // Создает список из 1М элементов
3   .filter { it > 10 } // Создает еще один список
4   .first()
```

Решение: `Sequence` — ленивая коллекция. Операции выполняются по одной на элемент.

```
1 val result = (1..1000000).asSequence()
2   .map { it * 2 }      // Ничего не вычисляется
3   .filter { it > 10 } // Ничего не вычисляется
4   .first()            // Вычисляется только до первого подходящего
```

Создание и преобразование Sequence

```
1 // Создание
2 val seq1 = sequenceOf(1, 2, 3)
3 val seq2 = (1..10).asSequence()
4
5 // Генератор
6 val fib = generateSequence(Pair(0, 1)) { (a, b) -> Pair(b, a + b) }
7     .map { it.first }
8     .take(10) // первые 10 чисел Фибоначчи
9
10 // Преобразование в коллекцию
11 val list = seq1.toList()
12 val set = seq1.toSet()
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.**
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

map и mapNotNull

```
1 val numbers = listOf(1, 2, 3, 4)
2
3 // map: преобразует каждый элемент
4 val doubled = numbers.map { it * 2 } // [2, 4, 6, 8]
5
6 // mapNotNull: отфильтровывает null после преобразования
7 val strings = listOf("1", "a", "3", "b")
8 val ints = strings.mapNotNull { it.toIntOrNull() } // [1, 3]
```

filter, filterNot, filterTo

```
1 val numbers = listOf(1, 2, 3, 4, 5)
2
3 // filter: оставляет, если условие true
4 val evens = numbers.filter { it % 2 == 0 } // [2, 4]
5
6 // filterNot: оставляет, если условие false
7 val odds = numbers.filterNot { it % 2 == 0 } // [1, 3, 5]
8
9 // filterTo: записывает результат в существующую коллекцию
10 val mutableList = mutableListOf<Int>()
11 numbers.filterTo(mutableList) { it > 3 } // mutableList = [4, 5]
```

take, drop, takeWhile, dropWhile

```
1 val numbers = listOf(1, 2, 3, 4, 5)
2
3 // take: первые N элементов
4 println(numbers.take(3)) // [1, 2, 3]
5
6 // drop: пропускает первые N
7 println(numbers.drop(3)) // [4, 5]
8
9 // takeWhile: берет, пока условие true
10 println(numbers.takeWhile { it < 4 }) // [1, 2, 3]
11
12 // dropWhile: пропускает, пока условие true
13 println(numbers.dropWhile { it < 4 }) // [4, 5]
```


reduce, fold, sum, maxOrNull

```
1 val numbers = listOf(1, 2, 3, 4)
2
3 // reduce: сворачивает список в одно значение, первый (элемент-начальный)
4 val sum1 = numbers.reduce { acc, n -> acc + n } // 10
5
6 // fold: сворачивается с явным начальным значением
7 val sum2 = numbers.fold(0) { acc, n -> acc + n } // 10
8
9 // sum: сумма чисел
10 val sum3 = numbers.sum() // 10
11
12 // maxOrNull: максимальный элемент или (null, если пусто)
13 val max = numbers.maxOrNull() // 4
```

groupBy, associate, partition

```
1 val words = listOf("a", "bb", "ccc", "dd")
2
3 // groupBy: группирует по ключу
4 val byLength = words.groupBy { it.length }
5 // {1=["a"], 2=["bb", "dd"], 3=["ccc"]}
6
7 // associate: создает Map по ключу и значению
8 val wordToLength = words.associate { it to it.length }
9 // {a=1, bb=2, ccc=3, dd=2}
10
11 // partition: разделяет на две части по условию
12 val (evens, odds) = numbers.partition { it % 2 == 0 }
13 // evens = [2, 4], odds = [1, 3, 5]
```

flatMap, flatten

```
1 val lists = listOf(listOf(1, 2), listOf(3, 4), listOf(5))
2
3 // flatten: объединяет вложенные списки
4 val flat = lists.flatten() // [1, 2, 3, 4, 5]
5
6 // flatMap: преобразует каждый элемент коллекции и объединяет
7 val words = listOf("Hello", "World")
8 val chars = words.flatMap { it.toList() } // [H, e, l, l, o, W, o, r, l, d]
```

distinct, sorted, sortedBy

```
1 val numbers = listOf(3, 1, 4, 1, 5)
2
3 // distinct: уникальные элементы
4 val unique = numbers.distinct() // [3, 1, 4, 5]
5
6 // sorted: сортировка по возрастанию
7 val sorted = numbers.sorted() // [1, 1, 3, 4, 5]
8
9 // sortedBy: сортировка по ключу
10 val words = listOf("bb", "a", "ccc")
11 val byLength = words.sortedBy { it.length } // [a, bb, ccc]
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takeIf**
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

let: безопасная работа с nullable

```
1 val str: String? = "Hello"
2
3 // let выполняется, только если объект не null
4 str?.let {
5     println(it.length) // 5
6     it.uppercase()      // возвращает результат блока
7 }
8
9 // Часто используется для цепочек
0 val result = str?.let { it.uppercase() } ?: "DEFAULT"
```

run: выполнить блок и вернуть результат

```
1 val result = run {  
2     val x = 10  
3     val y = 20  
4     x + y // возвращаемое значение  
5 }  
6 println(result) // 30  
7  
8 // Можно вызывать на объекте  
9 val str = "Hello"  
0 val length = str.run {  
1     println(this) // "Hello"  
2     length        // 5  
3 }
```

apply: конфигурация объекта

```
1 val list = mutableListOf<String>().apply {  
2     add("a")  
3     add("b")  
4     add("c")  
5 }  
6 // list = ["a", "b", "c"]  
7  
8 // Частодляинициализации  
9 val person = Person().apply {  
0     name = "Alice"  
1     age = 30  
2 }
```


takeIf и takeUnless: условное продолжение

```
1 val number = 15
2
3 // takeIf: возвращает объект , если условие true, иначе null
4 val evenOrNull = number.takeIf { it % 2 == 0 } // null
5
6 // takeUnless: возвращает объект , если условие false
7 val oddOrNull = number.takeUnless { it % 2 == 0 } // 15
8
9 // Комбинируем let
10 number.takeIf { it > 10 }
11     ?.let { println("Больше 10: $it") } // Больше 10: 15"
```

also: побочные эффекты без изменения

```
1 val list = listOf(1, 2, 3).also {  
2     println("Список создан: $it") // [1, 2, 3]  
3 }.map { it * 2 }.also {  
4     println("После map: $it") // [2, 4, 6]  
5 }  
6  
7 // Для логирования в цепочках  
8 val result = computeSomething()  
9     .also { log("Результат: $it") }  
0     .process()
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений**
- 7 Pair и Triple

Цепочка обработки с readLnOrNull()

```
1 fun main() {  
2     println("Введите число:")  
3     val number = readLnOrNull()  
4         ?.toIntOrNull()           // безопасное преобразование  
5         ?.takeIf { it > 0 }        // только положительные  
6         ?.let { "Результат: ${it * 2}" } // преобразование в строку  
7         ?: "Ошибка: введите положительное число "  
8  
9     println(number)  
0 }
```

Обработка списка чисел с ошибками

```
1 fun main() {
2     println("Введите числа через пробел :")
3     val sum = readlnOrNull()
4         ?.split(" ")                // разбиваем на части
5         ?.mapNotNull { it.toIntOrNull() } // преобразуем, отфильтровываем null
6         ?.filter { it > 0 }          // только положительные
7         ?.sum()                      // сумма
8         ?.takeIf { it < 1000 }       // проверка переполнения
9         ?: -1                       // ошибка
10
11     if (sum == -1) {
12         println("Ошибка: неверный ввод или сумма >= 1000")
13     } else {
14         println("Сумма: $sum")
15     }
16 }
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple**

Pair: пара значений

```
1 val pair = Pair("Hello", 42)
2 val (str, num) = pair // дефакто дефакто дефакто
3 println(str) // "Hello"
4 println(num) // 42
5
6 // Синтаксический сахар
7 val pair2 = "Hello" to 42
8
9 // Часто используется map
0 val map = mapOf("a" to 1, "b" to 2)
```

Triple: тройка значений

```
1 val triple = Triple("Alice", 30, "Engineer")
2 val (name, age, job) = triple
3 println("$name, $age, $job") // Alice, 30, Engineer
4
5 // Пример: возвращаем несколько значений из функции
6 fun getUserInfo(): Triple<String, Int, String> {
7     return Triple("Bob", 25, "Designer")
8 }
```


Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

Пример 1: Безопасный парсер координат

```
1 fun main() {  
2     println("Введите: x,y")  
3     val point = readlnOrNull()  
4         ?.split(",")  
5         ?.mapNotNull { it.trim().toDoubleOrNull() }  
6         ?.takeIf { it.size == 2 }  
7         ?.let { (x, y) -> "Точка: ($x, $y)" }  
8         ?: "Ошибка формата"  
9     println(point)  
0 }
```

Пример 2: Анализ частотности букв

```
1 fun main() {  
2     println("Введите текст:")  
3     val top3 = readlnOrNull()  
4         ?.filter { it.isLetter() }  
5         ?.groupingBy { it.lowercaseChar() }  
6         ?.eachCount()  
7         ?.toList()  
8         ?.sortedByDescending { it.second }  
9         ?.take(3)  
0         ?.joinToString { "${it.first}=${it.second}" }  
1         ?: "Нет данных"  
2     println("Топ-3 буквы: $top3")  
3 }
```

Пример 3: Валидация email-ов

```
1 fun isValidEmail(s: String) = s.count { it == '@' } == 1 &&  
2   s.split('@').let { it.size == 2 && it[0].isNotEmpty() && '.' in it[1] }  
3  
4 fun main() {  
5     println("Emailы' через ;")  
6     val valid = readlnOrNull()  
7         ?.split(";")  
8         ?.map { it.trim() }  
9         ?.filter { isValidEmail(it) }  
0         ?.ifEmpty { null }  
1         ?.joinToString("\n")  
2         ?: "Нет валидных email"  
3     println(valid)  
4 }
```

Пример 4: Генератор безопасных паролей

```
1 fun main() {  
2     val chars = ('A'..'Z') + ('a'..'z') + ('0'..'9')  
3     val pwd = (1..12).map { chars.random() }  
4         .joinToString("")  
5         .let { p -> if (p.any { it.isDigit() }) p else "Ошибка генерации" }  
6     println("Пароль: $pwd")  
7 }
```

Пример 5: Калькулятор выражений

```
1 fun eval(expr: String): Double? = try {
2     val tokens = expr.split(" ")
3     tokens.chunked(3).fold(tokens[0].toDouble()) { acc, part ->
4         when (part[1]) {
5             "+" -> acc + part[2].toDouble()
6             "-" -> acc - part[2].toDouble()
7             "*" -> acc * part[2].toDouble()
8             "/" -> acc / part[2].toDouble()
9             else -> return null
10        }
11    }
12 } catch (_: Exception) { null }
13
14 fun main() {
15     println("Введите: a op b op c ...")
16     val result = readlnOrNull()?.let(::eval)?.takeIf { it.isFinite() }
17     println("Результат: ${result ?: "Ошибка"}")
18 }
```

Пример 6: Анализ зависимостей

```
1 fun main() {  
2     println("Зависимости: A->B; B->C; ...")  
3     val deps = readlnOrNull()  
4         ?.split(";")  
5         ?.mapNotNull { it.split("->").takeIf { p -> p.size == 2 } }  
6         ?.associate { it[0].trim() to it[1].trim() }  
7         ?.let { map ->  
8             map.keys.filter { it !in map.values }  
9                 .isEmpty { listOf("Цикл!") }  
10                .joinToString()  
11        } ?: "Нет входных данных "  
2     println("Корневые: $deps")  
3 }
```


Пример 7: Преобразование единиц измерения

```
1 val units = mapOf("km" to 1000.0, "m" to 1.0, "cm" to 0.01, "mm" to 0.001)
2
3 fun main() {
4     println("Введите: значениеединицаиз _ -> единицав_")
5     val result = readlnOrNull()
6         ?.split(" ")
7         ?.let { (v, from, _, to) ->
8             (v.toDoubleOrNull() ?: return@let null) *
9             (units[from] ?: return@let null) /
10            (units[to] ?: return@let null)
11         }
12         ?.takeIf { it.isFinite() && it > 0 }
13         ?.let { "%.2f".format(it) }
14         ?: "Ошибка конвертации"
15     println("Результат: $result")
16 }
```

Пример 8: Построение маршрута

```
1 fun main() {  
2     println("Маршрут: A-B, B-C, C-D")  
3     val path = readlnOrNull()  
4         ?.split(", ")  
5         ?.map { it.split("-").takeIf { p -> p.size == 2 } }  
6         ?.filterNotNull()  
7         ?.fold("") { acc, (from, to) ->  
8             if (acc.isEmpty() || acc.last() == from[0]) acc + to[0]  
9             else return@fold "" // разрыв маршрута  
10        }  
11        ?.takeIf { it.length > 1 }  
12        ?: "Невалидный маршрут"  
13    println("Путь: $path")  
14 }
```

Навигация по лекции

- 1 Что такое функциональный стиль
- 2 Основные коллекции: List, Set, Map
- 3 Sequence: ленивые вычисления
- 4 Методы коллекций: map, filter, reduce и др.
- 5 Scope-функции: let, run, apply, takelf
- 6 Безопасная обработка без исключений
- 7 Pair и Triple

Итоги лекции

- Функциональный стиль — это иммутабельность, чистые функции, выражения
- List, Set, Map — основные коллекции; Sequence — для ленивых вычислений
- Основные методы: map, filter, reduce, groupBy, sorted, take, drop и др.
- Scope-функции: let, run, apply, takelf — для безопасной и читаемой работы с объектами
- Безопасная обработка: цепочки с ?. и ?: вместо try-catch
- Pair и Triple — для возврата нескольких значений
- Все примеры решены без var и mutable коллекций — как требуется на семинаре

Советы для семинара

- Начинайте с `readInOrNull()` — это ваша точка входа
- Используйте `?.` для безопасного доступа, `?:` для значения по умолчанию
- Разбивайте сложные цепочки на логические блоки (даже если пишете в одну строку)
- Тестируйте на граничных случаях: пустой ввод, `null`, ноль, отрицательные числа
- Помните: в Kotlin почти всё — выражение. Используйте это!

Удачи на семинаре!